

Glossaire & Q/R d'oral

B1 INFRA — Groupe 3 · Document de révision interne au tronc

Document à **lire avant l'oral**. Le glossaire reprend, en langage simple, tous les termes techniques mentionnés dans le projet. Les Q/R sont des questions probables du jury, avec une réponse **courte** à donner d'abord, puis un **développement** si on creuse. Les badges indiquent la difficulté de la question :

facile · moyenne · piège .

1. Glossaire — Réseau

Adresse IP

Identifiant numérique d'une machine sur un réseau (ex. `172.30.10.20`).

Sous-réseau / CIDR

Une plage d'IP définie par un préfixe.

`172.30.10.0/24` = 256 adresses, dont 254 utilisables.

Plan d'adressage

Document qui décrit qui prend quelle plage IP dans le réseau.

Routage

Décider, pour chaque paquet, par quelle interface l'envoyer.

Commutation

Aiguillage de trames Ethernet sur un switch, niveau 2 OSI.

VLAN

Réseau local **virtuel** : on découpe un même câble physique en plusieurs réseaux logiques. Niveau 2 OSI.

Segmentation réseau

Couper l'infra en zones isolées (RH, compta, technique, DMZ...) pour limiter ce qui se parle à quoi.

DMZ

Zone démilitarisée : la « façade » accessible depuis Internet (proxy, serveur web public). Tout ce qui est sensible est **derrière**.

DHCP

Service qui distribue automatiquement les IP aux machines qui arrivent sur le réseau.

DNS

Service qui traduit un nom

(`app.entreprise.local`) en IP. On a un DNS **interne** via dnsmasq pour notre zone.

Pare-feu

Filtre les paquets selon des règles (autoriser, refuser, journaliser).

nftables

Outil moderne de pare-feu sur Linux (remplace iptables). On l'utilise dans notre conteneur routeur.

iptables

L'ancienne génération de l'outil de pare-feu Linux. Caddy et Docker en posent automatiquement.

NAT / MASQUERADE

Translation d'adresses : on cache des IP privées derrière une IP publique. WireGuard et Docker en font.

WAN / LAN

WAN = grand réseau (Internet). LAN = réseau local.

OSI / TCP-IP

Modèles en couches. On parle souvent de la couche 2 (Ethernet/MAC), 3 (IP), 4 (TCP/UDP), 7 (HTTP, DNS...).

2. Glossaire — Web

HTTP

Protocole web en clair, port 80 par défaut.

HTTPS

HTTP + chiffrement TLS, port 443 par défaut.
Authentifie le serveur et chiffre l'échange.

TLS

Le mécanisme cryptographique qui sécurise HTTPS (anciennement SSL).

Certificat TLS

Pièce d'identité numérique du serveur, signée par une autorité (CA).

CA (autorité de certification)

L'entité qui signe les certificats. **CA interne** = on génère la nôtre (auto-signée). **Let's Encrypt** = CA publique gratuite.

SNI

« Server Name Indication » : le navigateur dit au serveur, dès le début du TLS, le nom qu'il cherche, ce qui permet de servir plusieurs sites HTTPS sur la même IP/port.

Reverse proxy

Serveur qui reçoit les requêtes des clients et les transmet à la bonne application interne (selon hôte, chemin, etc.).

Sous-domaine

Préfixe d'un domaine, comme `app.` ou `wordpress.` dans `entreprise.local`.

WordPress

CMS (système de gestion de contenu) écrit en PHP, avec base MySQL/MariaDB.

Node.js

Environnement d'exécution de JavaScript côté serveur.

MariaDB

Base de données relationnelle, dérivée de MySQL.

Caddy

Serveur web moderne avec gestion TLS automatique (Let's Encrypt et CA interne).

3. Glossaire — Sécurité & accès distant

VPN

« Virtual Private Network » : un tunnel chiffré entre deux machines à travers Internet, qui simule un câble réseau privé.

WireGuard

Le protocole VPN moderne qu'on a choisi : très simple, performant, chiffrement ChaCha20-Poly1305.

OpenVPN

VPN historique, plus complexe, basé sur OpenSSL. Alternative à WireGuard.

Paire de clés (public/privée)

Chaque peer a une clé privée (qu'il garde) et une publique (qu'il partage). Permet d'authentifier sans mot de passe.

PSK (clé pré-partagée)

Une clé secrète supplémentaire propre à chaque peer. Ajoute une couche d'authentification, et durcit le tunnel face à la cryptographie quantique.

Tunnel

Une connexion chiffrée qui transporte d'autres protocoles à l'intérieur (ex. ton HTTP voyage *dans* le tunnel VPN).

AllowedIPs

Liste des plages IP qu'on accepte de router via le tunnel. `0.0.0.0/0` = full-tunnel (tout passe par le VPN).

Endpoint

L'adresse publique (IP + port) du serveur VPN, vue depuis Internet.

Exposition minimale

Principe : n'ouvrir que les ports strictement nécessaires.

Moindre privilège

Principe : chaque service / utilisateur n'a que les droits dont il a besoin, rien de plus.

4. Glossaire — Virtualisation & conteneurs

Virtualisation

Faire tourner un système (OS, réseau) **simulé** par-dessus un autre. Outils : VirtualBox, VMware, Hyper-V.

Conteneur

Une application + ses dépendances empaquetées et isolées (mais qui partage le noyau Linux de l'hôte). Léger.

Image

Le modèle figé d'un conteneur (« recette ») — on instancie l'image en conteneurs.

Docker

Le moteur de conteneurs le plus répandu.

Docker Compose

Outil qui décrit une stack multi-conteneurs dans un fichier YAML — on lance tout d'un coup avec `docker compose up`.

Volume

Stockage persistant pour un conteneur (les données survivent à un redémarrage).

Bridge (réseau)

Réseau virtuel Docker par défaut, qui relie des conteneurs entre eux.

Réseau internal

Réseau Docker sans accès Internet sortant : utilisé pour notre zone data.

Bind mount

Monter un dossier de l'hôte dans le conteneur (utilisé pour le VPN `/opt/infra-vpn/wg-config`).

Reverse proxy en conteneur

Le proxy lui-même est dans un conteneur ; il route vers les autres conteneurs par leur nom de service.

5. Glossaire — Annuaire, partage, sauvegarde, admin

LDAP

Protocole et service d'**annuaire** : on y stocke des comptes utilisateurs, des groupes, etc. OpenLDAP est l'implémentation libre.

Active Directory

L'annuaire de Microsoft (équivalent commercial de LDAP + Kerberos).

DN / OU

DN = chemin unique dans l'annuaire (ex.

`uid=hugo,ou=people,dc=entreprise,dc=local`).

OU = organisational unit (dossier).

LDIF

Format texte pour décrire des entrées LDAP.

Notre `bootstrap.ldif` crée les comptes au

démarrage.

Samba

Implémentation libre du protocole SMB/CIFS de Microsoft : permet le partage de fichiers Windows-style.

SMB / CIFS

Le protocole derrière les

`smb://serveur/partage` dans le Finder ou l'Explorateur Windows.

Sauvegarde / restauration

Copier régulièrement les données critiques ; pouvoir les remettre en cas de pépin.

Rétention

Combien de versions on garde (chez nous : 10 dernières par type).

cron

Planificateur Unix : exécute une tâche selon une expression (ex. `* / 30 * * * *` = toutes les 30 min).

systemd

Le gestionnaire de services moderne sur Linux. Démarre / supervise les processus.

SSH

« Secure Shell » : connexion à distance chiffrée à une machine.

Git / GitHub

Versionnement du code et hébergement. Permet de retrouver l'historique exact des modifications.

Q/R — Questions probables du jury & réponses

Architecture & choix techniques

? Pourquoi Docker plutôt que VirtualBox, VMware ou Packet Tracer ? facile

✓ La grille accepte explicitement Docker comme outil de maquette. On a choisi Docker pour la **reproductibilité** (toute la stack se redéploie en une commande), pour le **versionnement** (tout est dans Git) et pour la **légèreté**. Si on avait voulu montrer de vrais VLAN de niveau 2 ou un pfSense, on aurait pris VMware ou Packet Tracer ; ici la segmentation logique nous suffisait.

? Pourquoi pas pfSense / OPNsense ? moyenne

✓ pfSense est une **distribution complète** de pare-feu, riche en fonctionnalités. C'est pertinent quand on a besoin d'un vrai routeur réseau dédié. Pour notre cas — segmenter logiquement et filtrer des flux — nftables dans un conteneur multi-zones est plus léger et tout aussi explicite. C'est aussi plus pédagogique : on voit les règles écrites en clair.

? Vos VLAN sont-ils de vrais VLAN ? piège

✓ Au sens strict (niveau 2 OSI avec tag 802.1Q), non : nos réseaux Docker sont des bridges isolés au niveau 3. Mais la segmentation **logique** est complète : chaque zone est sur un sous-réseau distinct, le pare-feu contrôle ce qui passe d'une zone à l'autre, et la zone data n'a aucune route sortante. C'est le résultat fonctionnel attendu par le brief, validé par la grille qui mentionne Docker.

? Pourquoi le port 18443 et pas 443 ? facile

✓ Pour ne pas entrer en conflit avec les autres services qui tournent sur le VPS. Sur une infra dédiée, on prendrait 443 directement.

? Et si le VPS tombe ? facile

✓ Tout est dans Git et tout est en Docker. Donc `git clone` + `docker compose up -d` sur n'importe quel hôte Linux avec Docker → la stack redémarre en quelques minutes. Les sauvegardes peuvent être restaurées dessus.

Réseau & pare-feu

? Comment garantit-on que la base n'est pas accessible depuis Internet ? facile

✓ Deux couches : (1) la base est sur un réseau Docker `internal: true` — elle n'a **aucune route** vers Internet ; (2) le conteneur n'a aucun port publié (`ports:` non défini) → impossible à atteindre depuis l'extérieur.

? **Que fait votre pare-feu nftables, concrètement ?** moyenne

✓ Dans le conteneur `infra-router-fw`, on charge une table `inet zones` avec un chain `forward` dont la **politique par défaut est drop**. Ensuite, on autorise explicitement les flux établis (`ct state established, related`), et un flux légitime entre deux interfaces. Tout le reste est jeté. C'est la posture « deny par défaut ».

? **Vous filtrez bien entre les zones ou juste à l'entrée ?** piège

✓ Le filtrage du conteneur routeur s'applique sur les flux **qui le traversent**. Pour les flux inter-conteneurs sur des bridges Docker distincts, Docker pose en plus ses propres règles iptables. La combinaison nous donne une isolation effective. Pour aller plus loin, on pourrait définir des *politiques* Docker plus strictes ou utiliser des plugins réseau.

Serveur web & TLS

? **Pourquoi Caddy plutôt que Nginx ou Apache ?** facile

✓ Parce que Caddy gère le TLS **automatiquement** : on déclare deux sous-domaines, et Caddy émet les certificats tout seul, soit via Let's Encrypt, soit via une CA locale (`tls internal`). C'est cinq lignes de configuration pour deux sites HTTPS. Avec Nginx, il faudrait gérer les certificats à part avec certbot.

? **Vos certificats sont auto-signés. En prod ?** facile

✓ En prod, on enlève `tls internal` et Caddy demande automatiquement les certificats à Let's Encrypt, à condition que les sous-domaines pointent vers le serveur. La configuration ne change quasiment pas.

? **Comment le proxy sait-il quel site afficher ?** moyenne

✓ Grâce au **SNI** dans le handshake TLS : le navigateur indique « je veux `app.entreprise.local` » dès l'ouverture de la connexion, avant même de chiffrer. Caddy lit cette information et sert le bon vhost. C'est le même mécanisme qu'un hébergement mutualisé.

? **Et pour faire du load balancing entre plusieurs Node ?** moyenne

✓ Caddy permet plusieurs *upstreams* dans un `reverse_proxy`. On lancerait 2 ou 3 conteneurs Node, on les listerait derrière la directive, et Caddy ferait du round-robin par défaut. C'est une extension naturelle de notre stack.

Sauvegarde & restauration

? **Vos sauvegardes sont-elles incrémentales ou complètes ?** facile

✓ Complètes (chaque fois on prend l'intégralité). Pour 25 Mo de WordPress et un dump SQL de 19 Ko, ça reste trivial. Pour un volume plus gros, on passerait sur un outil comme `restic` ou `borg` qui fait du déduplication incrémentale.

? **Comment garantissez-vous que la sauvegarde fonctionne ?** moyenne

✓ On a testé la restauration : un `restore.sh` prend un dump, le décompresse et le réinjecte dans la base. Tant qu'on n'a pas testé une restauration, on n'a pas une sauvegarde, on a un fichier d'archive.

? **Stratégie de rétention ?** moyenne

✓ Aujourd'hui : 10 dernières sauvegardes par type, donc environ 5 heures avec un run toutes les 30 min. En production, on appliquerait du « grand-père / père / fils » : par exemple 7 quotidiens, 4 hebdo, 12 mensuels.

? **Le stockage des sauvegardes est-il chiffré ?** piège

✓ Non. Le volume Docker est dans la zone data, donc isolé du réseau, mais les fichiers ne sont pas chiffrés au repos. Pour aller plus loin : `restic` chiffre nativement les sauvegardes ; ou un volume chiffré LUKS sur l'hôte. C'est une amélioration prévue.

VPN

? **Pourquoi WireGuard plutôt qu'OpenVPN ?** facile

✓ Trois raisons : (1) la configuration est **radicalement plus simple** (un fichier `.conf` de 10 lignes par client) ; (2) les performances sont meilleures ; (3) la surface d'attaque est plus faible — quelques milliers de lignes de code contre quelques centaines de milliers pour OpenVPN.

? **Qu'est-ce qui fait votre « authentification forte » ?** moyenne

✓ Chaque peer a sa propre paire de clés Curve25519 (publique/privée) **plus** une clé pré-partagée (PSK) unique. Le serveur n'accepte un client que si les deux concordent. Pas de mot de passe à voler, pas de phishing possible.

? **Vous gérez la révocation comment ?** moyenne

✓ On retire le bloc `[Peer]` de la conf serveur et on redémarre WireGuard. Le client perd l'accès immédiatement. Avec notre image conteneurisée, on retire le nom de la variable d'environnement `PEERS` et on recrée le conteneur.

? **Le VPN est-il « full-tunnel » ou « split-tunnel » ?** piège

✓ Full-tunnel : `AllowedIPs = 0.0.0.0/0` du côté client → **tout** son trafic Internet passe par le tunnel. C'est ce qu'on veut pour un télétravailleur : son DNS et son trafic sont protégés.

Bonus LDAP & Samba

? **Samba est-il authentifié par LDAP ?** piège

✓ Pas dans cette version : Samba a ses propres comptes, mais les UIDs sont alignés (10001 pour Hugo, etc.) avec LDAP. L'intégration directe se fait avec le back-end `ldapsam` ou via SSSD/winbind ; c'est notre piste d'évolution.

? **Comment on ajoute / supprime un utilisateur LDAP ?** facile

✓ Ajout : on rajoute un bloc dans `bootstrap.ldif` et on relance ; ou à chaud, avec `ldapadd` en se branchant sur l'annuaire avec les credentials admin. Suppression : `ldapdelete`.

? **Le partage Samba est-il accessible depuis Internet ?** facile

✓ Non, et c'est volontaire — Samba sur Internet est une mauvaise idée. Le conteneur n'a aucun port publié ; il faut être **sur le VPN** ou sur le LAN de l'entreprise pour y accéder.

Docker, sécurité globale, secrets

? **Comment gérez-vous les secrets (mots de passe) ?** piège

✓ Ici, en clair dans le `docker-compose.yml` pour la démo — c'est conscient et documenté. En production, on utilise les **secrets Docker** ou un fichier `.env` non versionné, idéalement un coffre type Vault ou les *secrets* du fournisseur cloud.

? **Docker est-il une vraie isolation pour la sécurité ?** piège

✓ L'isolation est forte mais pas absolue (les conteneurs partagent le noyau). Pour des charges très sensibles, on ajouterait des profils AppArmor/seccomp, on tournerait en *rootless*, ou on passerait sur des VM légères type Firecracker / Kata.

? **Et si quelqu'un compromet un conteneur ?** moyenne

✓ Grâce à la segmentation : un attaquant qui prend le WordPress n'atteint pas la base directement par le réseau public (zone data *internal*). Il peut taper sur la base via les credentials applicatifs, mais il ne peut pas exfiltrer en sortie (pas de route Internet). Bonus : on ajouterait des règles de sortie au pare-feu pour le proxy aussi.

Méthode & doc

? Comment vous êtes-vous répartis le travail à 3 ? facile

✓ Hugo sur l'architecture, le réseau et le VPN. Shakil sur le serveur web et le reverse proxy. Mathéo sur la sauvegarde et le bonus LDAP/Samba. La doc et la grille en commun.

? Où est votre documentation ? facile

✓ Tout est dans le dépôt GitHub [H129hj/infra-entreprise-b1](https://github.com/H129hj/infra-entreprise-b1) : README, doc technique, schéma réseau, support oral, captures de preuves, et même les scripts de génération du diaporama.

? Qu'est-ce que vous feriez avec plus de temps ? moyenne

✓ Cinq pistes : (1) intégration Samba ↔ LDAP avec *Idapsam* ; (2) un vrai domaine + Let's Encrypt ; (3) du monitoring (Prometheus + Grafana) ; (4) du load balancing actif sur Caddy ; (5) du chiffrement au repos des sauvegardes via *restic*.